

Platform Architecture

Platform Architecture — Gym, Office & Academy Management

1. Architecture Principles
2. High-Level System Architecture
3. Multi-Tenancy Model
4. Core Domain Model (Shared Primitives)
5. Module Architecture
 - 5.1 Gym module
 - 5.2 Office module
 - 5.3 Academy module
6. Entity Relationship Overview
7. Recommended Tech Stack
8. API Architecture
9. Auth, Roles & Permissions
10. Notifications Architecture
11. Deployment & Infrastructure
12. Non-Functional Requirements

Platform Architecture — Gym, Office & Academy Management

System & Data Architecture reference for a multi-vertical management platform: shared core primitives, per-vertical modules, data model, tenancy model, and a recommended tech stack.

Assumption: no specific tech stack, cloud provider, or deployment target was specified, so this document uses a common, cost-efficient stack as a reference recommendation (Section 7) while keeping the architecture and data model framework-agnostic. Swap in your preferred stack — the module boundaries and data model don't change.

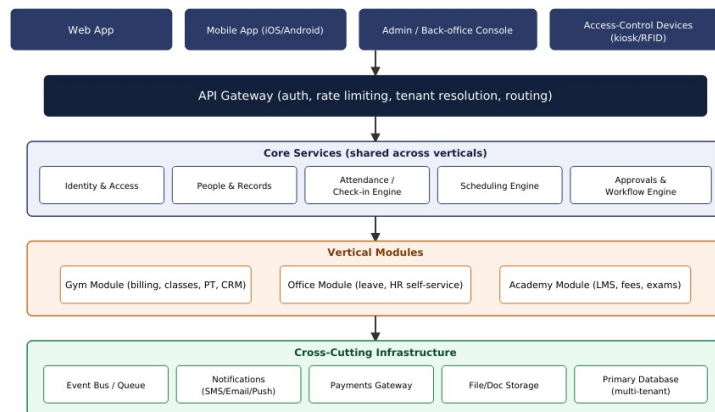
1. Architecture Principles

1. **Shared core, thin verticals** — build the primitives that repeat across gym/office/academy once (people, attendance, approvals, scheduling, billing, notifications, audit), then layer vertical-specific modules on top instead of three separate codebases.
2. **Multi-tenant by default** — every table and query is scoped to a tenant (a gym chain, a company, an academy) from day one, not retrofitted later.
3. **Module boundaries, not microservices (initially)** — start as a modular monolith with clear internal boundaries between core and verticals; split into services only when a specific module's scale or team ownership actually demands it.
4. **Events over tight coupling** — cross-module effects (e.g. attendance affecting billing, or an approval triggering a notification) go through an internal event bus, not direct cross-

module function calls.

5. **API-first** — the web app, mobile app, and any future integrations all consume the same versioned API; no logic lives only in the frontend.
6. **Audit everything that changes state** — attendance edits, approval decisions, billing changes, and role changes are all append-only logged.

2. High-Level System Architecture



System architecture

Requests come in through a single API gateway that resolves the tenant, authenticates the caller, and routes to the appropriate service. Core services are shared by all three verticals; vertical modules sit on top of them and add domain-specific logic (billing rules for gyms, leave rules for offices, gradebooks for academies). Everything that produces a side effect (a notification, a billing event) goes through the event bus rather than calling another module directly.

3. Multi-Tenancy Model

Each customer (a gym chain, a company, an academy) is a **tenant**. Two viable strategies, in order of recommendation for this platform's likely scale:

Strategy	How it works	Trade-off
Shared database, tenant_id column (recommended to start)	Every tenant-scoped table carries a <code>tenant_id</code> ; every query is filtered by it, enforced at the ORM/query-builder layer and backed by row-level security	Simplest to operate and cheapest at low-to-mid scale; requires strict discipline to never leak a query without the tenant filter
Schema-per-tenant	Each tenant gets its own database schema	Stronger isolation, but migrations and cross-tenant reporting get harder as tenant count grows

Database-per-tenant	Each large tenant gets a dedicated database	Reserved for enterprise/large tenants only, once the platform has them
---------------------	---	--

Recommendation: start with shared-database + tenant_id and row-level security. It's the standard approach for SaaS at this stage and keeps operational cost low; revisit only if a specific large customer requires dedicated isolation.

A tenant additionally declares which **modules** it has active (gym / office / academy — a customer could in principle run more than one, e.g. a school that also manages its own staff HR), which drives which vertical tables and UI surfaces are exposed to it.

4. Core Domain Model (Shared Primitives)

These are the tables/entities built once and reused by every vertical, as identified in the feature research:

Primitive	Purpose	Used by
tenant	The customer account; owns all data	All
user / role / permission	Identity, authentication, role-based access control	All
person	A generic person entity with a type (member / employee / student) and lifecycle status	All
attendance_record	Proof-of-presence event: person, timestamp, method (biometric/QR/RFID/manual), location	Gym, Office, Academy
approval_request	A generic routable request: type, requester, routing chain, current status, decision log	Office (leave, expense), Academy (lea
schedule_slot	A time-and-capacity object: start/end time, capacity, resource (room/trainer/teacher)	Gym (classes/PT), Academy (batches/peri
booking	A person's reservation against a schedule_slot, with status (confirmed/waitlisted/cancelled/no-show)	Gym, Acaden
invoice / payment	Billing: recurring or one-off charges, payment records, refunds	Gym, Acaden
notification	Outbound message record: channel, recipient, template, status	All
audit_log	Append-only record of state changes: who, what, when, before/after	All
document	Uploaded file metadata (report cards, payslips, contracts) with access control	Office, Acade

5. Module Architecture

5.1 Gym module

Extends the core with gym-specific entities and logic: `membership_plan` (billing interval, billing day, commitment period), `membership` (a person's active plan instance), `class` and `pt_package` built on `schedule_slot`, `lead` and `pipeline_stage` for the sales funnel, and `churn_signal` computed from attendance gaps. Billing logic (proration, freeze/pause, dunning) lives here, calling the shared invoice/payment primitives.

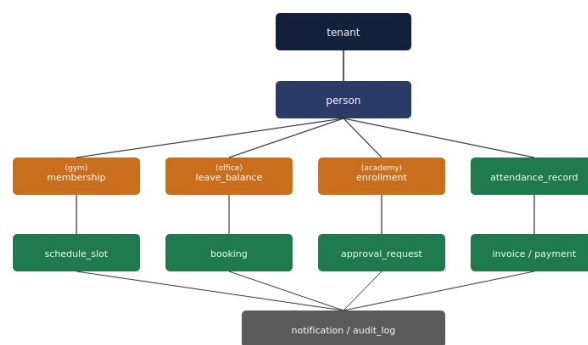
5.2 Office module

Deliberately thin, per the corrected scope: `leave_type` and `leave_balance` extend `person`; leave applications and expense claims are just `approval_request` records with a specific type; attendance uses the shared `attendance_record` directly with no gym/academy-specific fields. No scheduling, billing, or booking entities are needed here — this module is the smallest of the three.

5.3 Academy module

The most feature-dense module: `course`, `batch` (built on `schedule_slot`), `enrollment`, `guardian` linked to `person`, `assignment` and `submission`, `exam` and `gradebook_entry`, `fee_plan` and `installment` (extending the shared invoice primitive). Attendance again reuses the shared `attendance_record`, with academy-specific parent-alert logic subscribing to attendance events on the event bus rather than being hard-coded into the core.

6. Entity Relationship Overview



Entity relationship overview

Legend: navy = tenancy/identity core, blue = person core, orange = vertical-specific extensions of person, green = shared cross-vertical primitives, grey = system-wide logging/messaging that every other entity feeds into.

7. Recommended Tech Stack

This is a reference recommendation, not a requirement — chosen for fast iteration, a small team, and low initial infra cost, all of which fit a YC-track early-stage build. Substitute freely.

Layer	Recommendation	Why
Frontend (web)	Next.js (React) + TypeScript	Fast iteration, SSR where useful, large ecosystem, one language across frontend/backend if paired with Node
Mobile	React Native	Shares logic/types with the web frontend; one codebase for iOS + Android
Backend API	Node.js (NestJS) or alternatively Python (FastAPI)	NestJS gives you modular boundaries out of the box, which maps directly onto the core/vertical module split in Section 5
Primary database	PostgreSQL	Row-level security supports the tenant-isolation model directly; strong relational fit for the entity model in Section 4
Cache / queue	Redis (cache) + a queue (BullMQ or SQS)	Backs the event bus and background jobs (billing runs, notification sends, report generation)
File storage	S3-compatible object storage	Documents, report cards, payslips, uploaded IDs
Auth	Own auth service or Auth0/Clerk, backed by JWT + refresh tokens	Buy vs build early is a reasonable trade-off here; revisit if enterprise SSO becomes a requirement
Notifications	Twilio (SMS/WhatsApp) + SendGrid/SES (email) + Firebase Cloud Messaging (push)	Covers all channels named in the feature research
Payments	Stripe (or a regional gateway if targeting a specific market first)	Handles recurring billing, proration, and refunds directly — reduces custom billing-engine work

Hosting	A managed platform (Railway/Render/Fly.io) early, migrating to AWS/GCP as scale demands	Minimizes DevOps overhead for a 3-person team
Observability	Sentry (errors) + a basic metrics/log stack (e.g. Grafana Cloud)	Cheap to start, sufficient for early scale

8. API Architecture

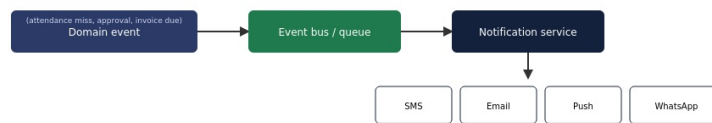
- **REST, versioned** (/api/v1/...) — simplest to build against from web and mobile alike; move to GraphQL only if the client-side over/under-fetching problem actually shows up
- **Tenant resolution** — via subdomain ({tenant}.platform.com) or an explicit header/token claim, resolved once at the gateway and passed down as request context
- **Consistent resource shape** across verticals — e.g. GET /people, GET /attendance, GET /approvals behave the same way regardless of which module the caller is in, with vertical-specific fields nested rather than bolted onto the core shape
- **Idempotency keys** on write endpoints that trigger billing or notifications, to make retries safe
- **Webhooks out** — for future integrations (accounting software, external calendars), the platform should be able to emit events, not just receive them

9. Auth, Roles & Permissions

Role tier	Scope	Example
Platform admin	Cross-tenant, internal only	Zhilo Labs support/ops staff
Tenant admin	Full access within one tenant	Gym owner, HR head, academy principal
Manager / staff	Scoped to a team, department, or batch	Trainer, department manager, class teacher
End user	Self-service only — own records	Gym member, employee, student/parent

Permissions are role-based (RBAC) at the module level (e.g. office.leave.approve, gym.billing.view) rather than hard-coded per-endpoint, so new roles can be composed from existing permissions as the platform grows. Every approval decision, role change, and billing edit writes to audit_log.

10. Notifications Architecture



Notifications architecture

Notifications are decoupled from the modules that trigger them. A gym billing failure, an office leave approval, or an academy absence alert all just emit a domain event; the notification service decides channel, template, and delivery, and logs delivery status independently of the originating module.

11. Deployment & Infrastructure

- **Start as a modular monolith** — one deployable backend with clear internal module boundaries (Section 5), one Postgres database with tenant-scoped rows. This is the fastest path for a 3-person team and avoids premature microservice overhead.
- **Background workers** — a separate worker process (same codebase, different entrypoint) handles billing runs, scheduled reminders, report generation, and notification delivery, so these don't block API request latency.
- **Environments** — dev, staging, production, each with isolated databases; staging mirrors production config for safe testing of billing/notification flows especially.
- **CI/CD** — automated tests + migrations gate every deploy; database migrations are backward-compatible (expand/contract pattern) to avoid downtime.
- **When to split out a service** — only when a specific module (most likely billing, given its complexity and audit requirements) needs independent scaling, a different release cadence, or a dedicated team.

12. Non-Functional Requirements

Concern	Approach
Data isolation	Tenant-scoped rows + row-level security; no query path that can cross tenants
Privacy (biometric data)	Store biometric templates (not raw images) where possible, encrypted at rest, with an opt-out/non-biometric alternative per the earlier feature research
Auditability	Append-only audit_log for attendance edits, approvals, billing changes, and role changes
Availability	Attendance/check-in must degrade gracefully offline (buffer-and-sync, as noted in the gym/academy research) since it's the most latency- and uptime-

	sensitive path
Scalability	Stateless API layer behind a load balancer; database read replicas once reporting/analytics load grows
Compliance	Data residency and consent handling (especially for biometric and student/minor data) should be reviewed per target market before launch

Prepared as a technical architecture reference for early-stage platform planning. Stack and infrastructure choices in Sections 7 and 11 are recommendations, not fixed requirements — the module boundaries and data model in Sections 4-6 are the parts intended to hold regardless of implementation choices.